

Python para el 'Ninja' de Django

Guía visual de correlación: De los fundamentos de Python a la arquitectura web profesional.



El Gran Panorama: La Arquitectura MVT

¿Cómo funciona Django? Imagina un restaurante de alta cocina.



Tu Mapa Ninja: Tabla de Correlación Rápida

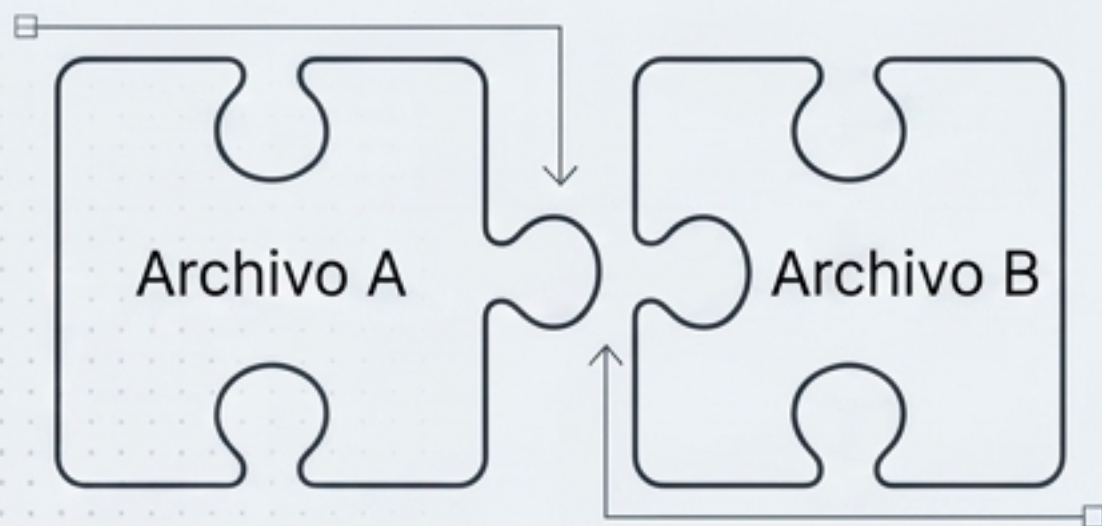
Concepto Python	Uso en Django	Archivo Clave
 Clases (POO)	Definir tablas de base de datos	<code>models.py</code>
 Funciones	Procesar solicitudes (El Chef)	<code>views.py</code>
 Diccionarios	Enviar datos a la interfaz (El Menú)	<code>views.py -> templates</code>
 Listas	Mostrar múltiples registros	<code>templates/lista.html</code>
 Operadores <code>==</code>	Validar métodos POST o GET	<code>views.py</code>
 Importaciones	Reutilizar herramientas externas	Todos los archivos

1. Importaciones y Modularidad (El Esqueleto)

Concepto Python

Ensamblando Piezas

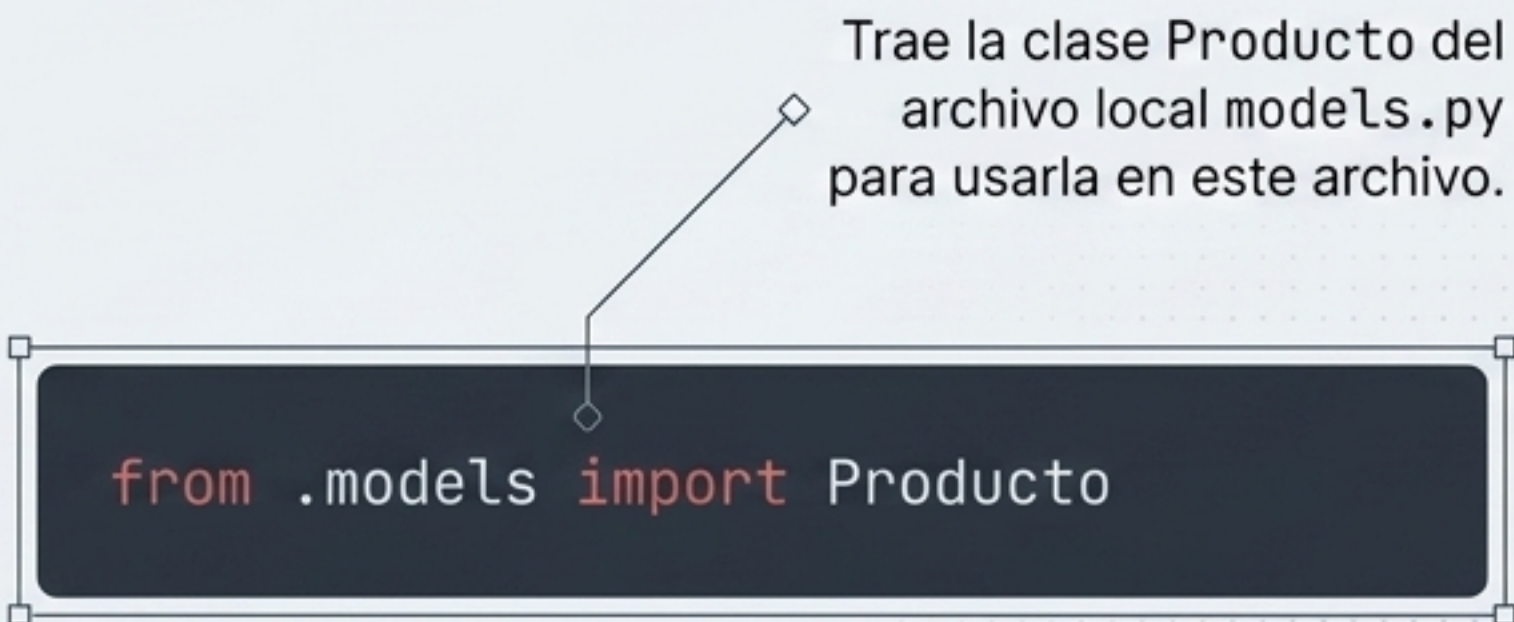
Uso de `import` y `from ... import ...` para reutilizar código de otros archivos. Python necesita saber exactamente dónde buscar las herramientas.



En Django

Conectando el Framework

Se usa para traer modelos a las vistas o importar herramientas de seguridad.



2. Clases y Herencia (El Corazón: Modelos)

Concepto Python

Programación Orientada a Objetos (POO).

Una class es un molde.

La herencia permite que una clase "hija" obtenga poderes de una clase "padre".

- 1. El Molde:

```
class Estudiante(models.Model):
```
- 2. Las Columnas:

```
nombre = models.CharField(max_length=100)
```
- 3. La Representación:

```
def __str__(self):
```

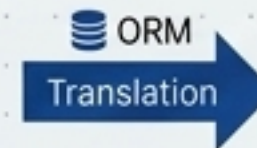
(Determina cómo se ve el objeto como texto).

En Django

Django utiliza POO **para definir la base de datos sin escribir SQL.**

Todos los modelos heredan de `models.Model`.

```
class Producto(models.Model):  
    nombre = models.CharField(max_length=100)  
    precio = models.DecimalField(...)  
    stock = models.IntegerField()
```

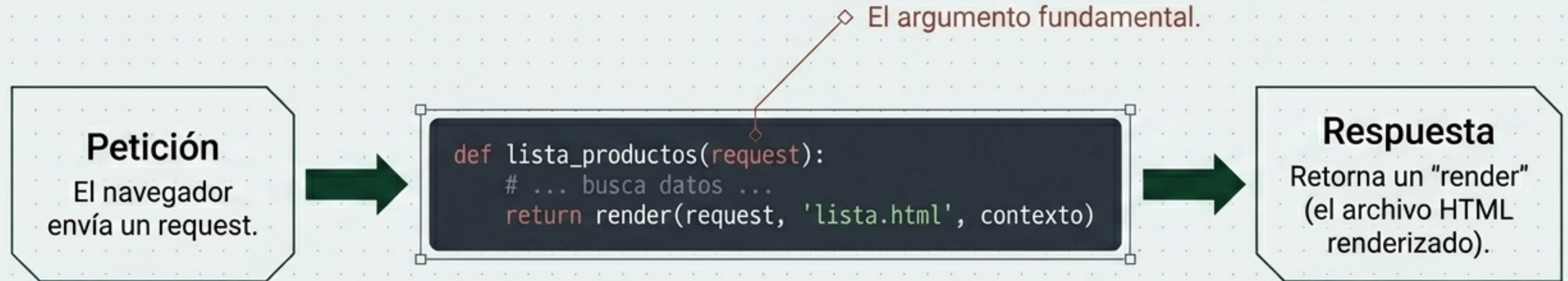


app_producto			
id	nombre	precio	stock
1	Laptop	999.99	15

3. Funciones y Argumentos (La Lógica: Vistas)

Las funciones (def) reciben argumentos, procesan información y retornan un valor.

Las Vistas Basadas en Funciones (FBV) siempre reciben un objeto de entrada llamado request.



Evolución del Ninja: FBV vs CBV

Vistas por Funciones (FBV)

Simple y Explícitas. Ideales para principiantes y lógica altamente personalizada. Todo el flujo es visible.

```
def mi_vista(request):  
    data = Producto.objects.all()  
    return render(request, 'lista.html', {'data': data})
```



Ideal para principiantes, lógica personalizada.



Requiere más líneas de código (repetitivo).

Vistas por Clases (CBV)

Potentes y Reutilizables. Aplican la herencia avanzada de Python (ListView, DetailView, CreateView).

```
class ProductoList(ListView):  
    model = Producto  
    template_name = 'lista.html'
```

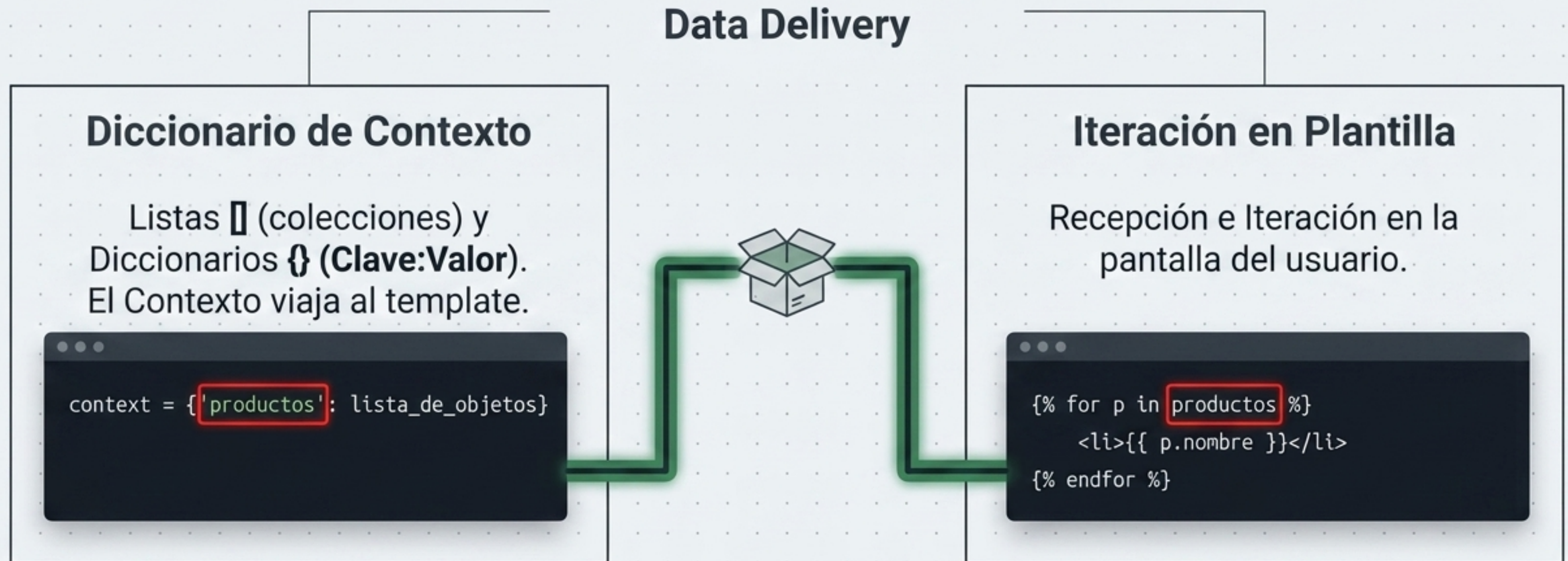


Menos código (DRY). Lógica común ya construida por Django.



Curva de aprendizaje mayor.

4. Diccionarios y Listas (La Capa de Presentación)



5. Decoradores (Lógica y Seguridad)

Un **decorador modifica el comportamiento de una función** sin cambiar su código interno. Se coloca una capa por encima usando @.

Protección profesional “Out of the Box”. Se usa para proteger páginas que requieren inicio de sesión.

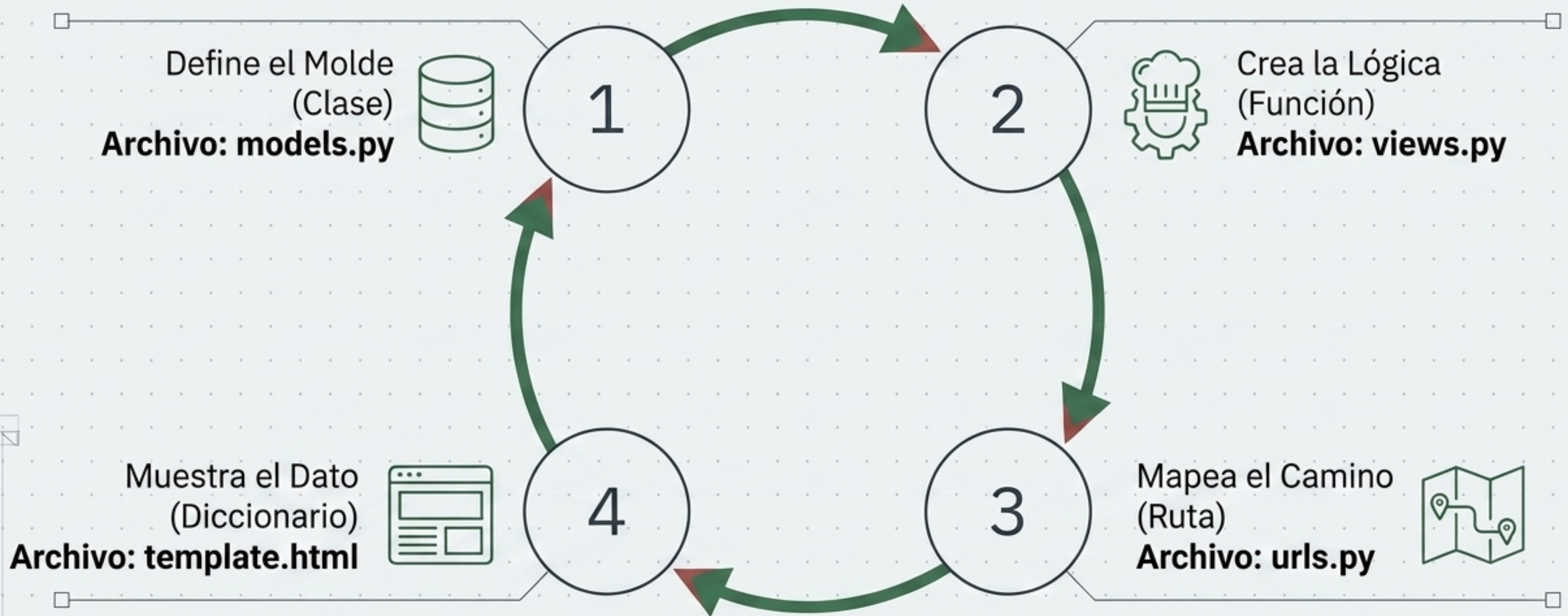


Actúa como un portero: impide el acceso al bloque de código inferior si el usuario no está autenticado.

```
from django.contrib.auth.decorators import login_required  
  
@login_required  
def dashboard(request):  
    # ...
```


Síntesis: El Flujo "Paso a Paso"

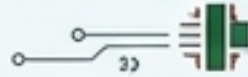
Para no perderte, recuerda que el desarrollo de una nueva funcionalidad siempre sigue este circuito cerrado:



Implementación en Plantillas: Lógica Python en HTML

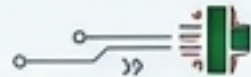
Condicionales de Estado

Python evalúa si el usuario inició sesión para mostrar opciones ocultas.



Seguridad Integrada

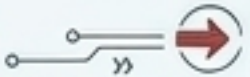
Etiqueta obligatoria para proteger formularios contra ataques de falsificación.



```
<html lang="en">
<head>
</head>
<body>
  <header> Estudiantes </header>
  <nav>
    {% if user.is_authenticated %}
      Hola, {{ user.username }}!
      <form method="post" action="{% url 'logout' %}" style="display: inline-block; vertical-align: middle; margin-left: 10px;">
        {% csrf_token %}
        <button type="submit" style="border: none; background: none; padding: 0 10px;">
          Cerrar Sesión
        </button>
      </form>
    {% else %}
      <a href="{% url 'login' %}">Iniciar Sesión</a>
    {% endif %}
  </nav>
  <main>
    {% block content %}
    {% endblock content %}
  </main>
</body>
</html>
```

Inyección de Variables

La doble llave extrae valores del diccionario de contexto y los imprime en pantalla.



La Consola del Ninja: Operaciones CRUD con Python

El ORM (Object-Relational Mapping) te permite hablar con la base de datos usando Python puro en lugar de código SQL.

```
>>> Producto.objects.all()
# Obtener todos los registros.

>>> Producto.objects.filter(precio__lt=100)
# Filtrar registros (precio menor a 100).

>>> Producto.objects.create(nombre='Laptop', precio=999.99)
# Crear e insertar un nuevo registro instantáneamente.
```

Todo este código de Python se traduce automáticamente a comandos SQL seguros por debajo de la mesa.

Reto Práctico: Diagnóstico de Código

Encuentra los 3 errores conceptuales de Python en esta vista de Django.

```
def guardar_datos(request):  
    if request.method = 'POST':  
        # lógica de guardado...  
  
    return render('formulario.html')
```



Lista de Control (Auditoría)

- ☐ ¿Sintaxis de función correcta? (Falta ':' en def)
- ☐ ¿Uso correcto de operadores de validación? ('=' vs '==')
- ☐ ¿Argumentos completos en la respuesta? (Falta 'request' en render)

El Checklist del Ninja

Eres un desarrollador web en Django cuando entiendes que:



No hay magia negra: Todo el framework es solo código Python modular bien organizado.



Tus clases (models.py) son el esqueleto que estructura tus datos.



Tus funciones (views.py) son los cerebros que deciden qué hacer con esos datos.



Tus diccionarios (context) son los vehículos que transportan la información a la pantalla del usuario.

Con los fundamentos de Python claros, la arquitectura web es solo cuestión de práctica.